PROJECT DOCUMENTATION

Three HTML Webpages

AccessHub • Orbit • BusGo

Prepared from the three supplied HTML files. This document describes the implemented front-end interfaces and identifies prototype limitations; it does not assume external services or back-end systems exist.

1. Introduction

This document describes three independent webpage prototypes: account access, project/task management, and bus seat selection. The pages use HTML for structure, CSS for presentation, and JavaScript for client-side interactions.

2. Project Objectives

• Describe the purpose and main functions of each webpage.

• Explain the typical user workflow in simple steps.

• Identify front-end validation and interactive behaviour present in the code.

• Record limitations that must be addressed before production use.

• Provide a basic manual testing checklist for each page.

3. Technologies and Implementation

• HTML: page structure, forms, controls, dialogs, and sections.

• CSS: layout, typography, colours, spacing, cards, responsive layouts, and visual states.

• JavaScript: form handling, view switching, task creation/filtering, drag-and-drop, seat selection, fare calculation, and interface messages.

• The supplied files can be opened directly in a modern browser; no build step is described in the files.

1. AccessHub — Login & Register

Source file: ABL 1.html

Purpose

A student account interface that presents login, account registration, and password-recovery views in one webpage.

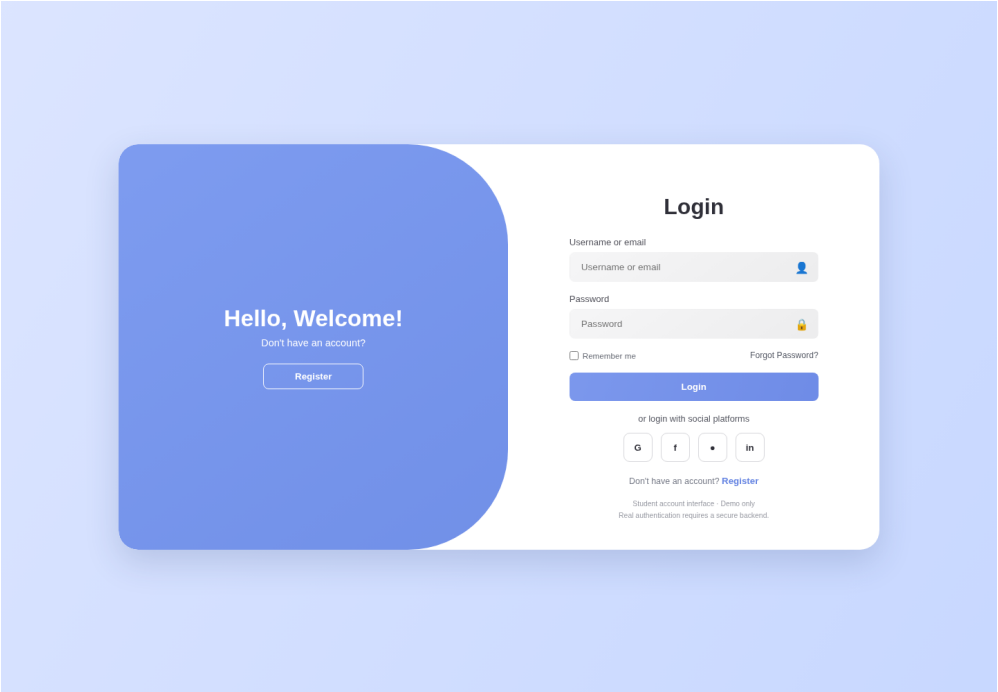


Figure: Screenshot of the webpage interface

Main Features

• Login form with email and password fields.

• Registration form with full name, email, password, and confirm-password fields.

• Password-recovery form that accepts a registered email address.

• Navigation buttons switch between Login, Register, and Forgot Password views.

• JavaScript checks the email format, requires a password of at least eight characters containing a letter and a number, checks matching passwords, and prevents duplicate demo email registrations.

• Demo user records are stored in browser localStorage under the key accessHubDemoUsers.

User Workflow

1. Open the HTML file in a modern browser.

2. Choose Login to enter existing demo credentials, or choose Register to create a demo account.

3. Enter the requested details and submit the form.

4. Use Forgot Password to open the recovery form.

Limitations and Notes

• The page itself labels the account interface as demo-only.

• Browser localStorage is not a secure user database; passwords must not be stored this way in a real service.

• Password recovery is a front-end demonstration and does not send a real email or reset a server-side password.

• The social sign-in buttons are demo interface elements, not verified OAuth integrations.

Suggested Manual Tests

• Try submitting registration with an invalid email.

- Try a password shorter than eight characters or without both a letter and a number.
- Enter different values in Password and Confirm password.
- Register the same email twice and confirm the duplicate check appears.
- Switch among Login, Register, and Forgot Password views.

2. Orbit — Project Workspace

Source file: ABL 2.html

Purpose

A project-management dashboard for viewing project cards and managing tasks across a Kanban-style workflow.

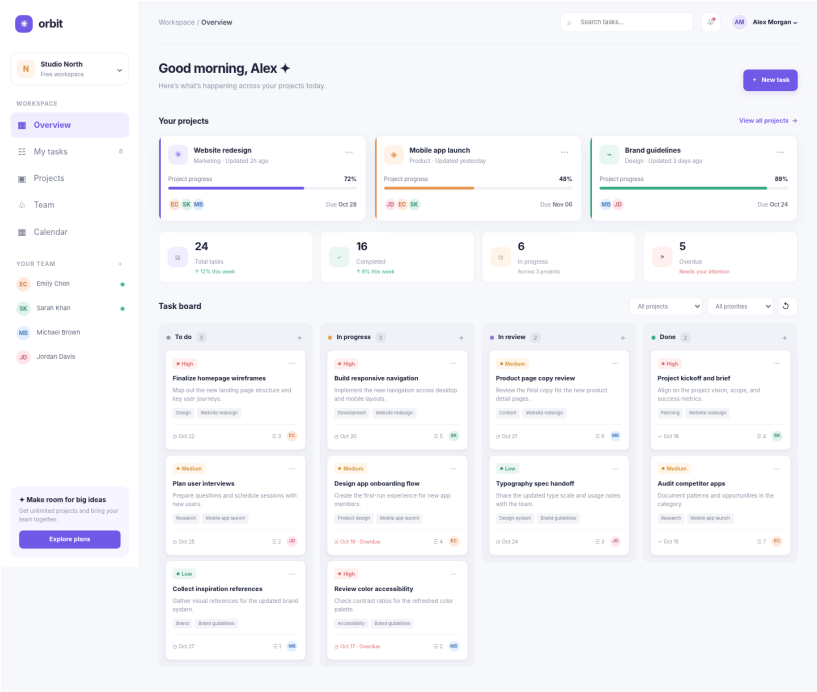


Figure: Screenshot of the webpage interface

Main Features

• Workspace navigation and a project overview area.

• Project cards, including Website redesign, Mobile app launch, and Brand guidelines.

• Task board with To do, In progress, In review, and Done columns.

• Create-task dialog and form for adding tasks.

• Search and filters for narrowing the visible task list by project and priority.

• Task counts are recalculated from the task cards.

• Drag-and-drop interactions allow tasks to move between board columns.

• Toast messages provide brief feedback for supported actions.

User Workflow

1. Open the HTML file in a modern browser.

2. Review the project cards and task board.

3. Use the task-creation control to open the task form and submit task details.

4. Use search, project filter, or priority filter to narrow tasks.

5. Drag a task card to another workflow column when supported by the browser.

Limitations and Notes

• The supplied page is a front-end prototype; no server or shared database is included in the HTML file.

• Task changes should be treated as local demo state unless persistence is added separately.

• Workspace controls that visually resemble account, upgrade, or navigation actions should not be assumed to connect to a real service.

Suggested Manual Tests

- Create a task with valid details.
- Search for a task by a word in its title.
- Change the project and priority filters.
- Move a task between columns and check the task counts.
- Try the form with required fields left blank.

3. BusGo — Find Your Seat

Source file: ABL 3.html

Purpose

A bus-travel booking interface prototype that lets a user view a journey, choose a coach, select seats, and enter passenger and journey details.

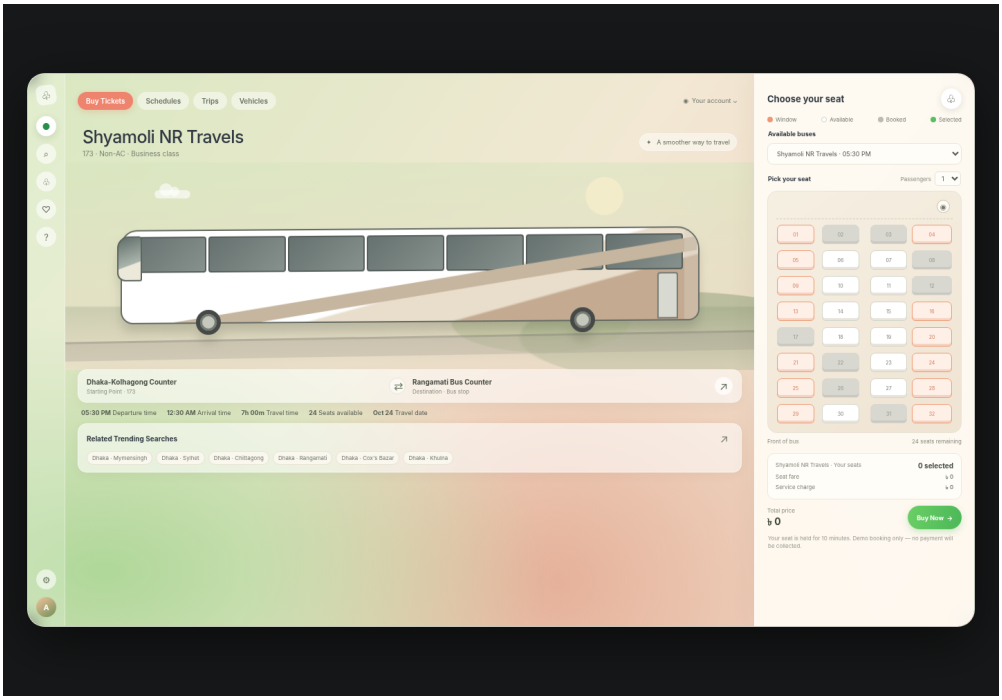


Figure: Screenshot of the webpage interface

Main Features

• Travel interface with route/journey information and navigation views such as Tickets, Schedules, Trips, and Vehicles.

• Coach selection using the available bus options in the page's JavaScript data.

• A 32-seat layout generated dynamically by JavaScript.

• Booked seats are disabled; selected seats are highlighted.

• Passenger count limits how many seats can be selected.

• Fare, service fee, and total are updated based on the selected seats.

• Passenger-details and journey-edit dialogs.

• Route swapping, date changes, saved-route/bookmark interaction, and notification messages are wired in the front-end code.

User Workflow

1. Open the HTML file in a modern browser.
2. Review the displayed journey and choose a coach if needed.
3. Set the passenger count and select available seats.
4. Review the fare summary.
5. Use the booking control to open passenger details and complete the demo form.
6. Use the journey-edit controls to change route/date in the prototype.

Limitations and Notes

• Bus and booked-seat information is hard-coded demonstration data in the page.

• The interface does not, by itself, confirm seats with a real bus operator or maintain a central booking inventory.

• No real payment gateway, ticket issuance, or backend booking API is included in the supplied HTML.

• The displayed fare format uses the ■ symbol in the current implementation; confirm and localize the currency before real deployment.

Suggested Manual Tests

• Select and deselect an available seat.

• Try selecting more seats than the passenger count permits.

• Confirm booked seats cannot be selected.

• Change coach and verify the seat layout/fare updates.

• Change the passenger count and check the selected-seat limit.

• Open and close the passenger and journey dialogs.

5. How to Run the Webpages

1. Keep each HTML file in a folder on your computer.

2. Double-click the required HTML file or open it using a modern browser such as Chrome or Edge.

3. Test the visible controls and forms using sample data only.

4. If publishing on GitHub, upload each HTML file and the documentation. Ensure screenshot paths match the repository folders.

6. General Recommendations Before Production

• Add a secure backend and database for accounts, tasks, trips, passengers, and bookings where required.

• Never store real passwords in browser localStorage. Use server-side password hashing, secure sessions, and established authentication flows.

• Add server-side validation in addition to front-end validation.

• For BusGo, integrate live schedules, operator inventory, payment processing, booking confirmation, cancellation rules, and ticket generation before claiming real booking capability.

• For Orbit, add persistent storage, user permissions, audit history, and conflict handling if multiple users will work on the same board.

• Test responsive behaviour, keyboard access, screen-reader labels, error states, and common browser sizes.

7. Conclusion

The three files provide useful front-end demonstrations of common web application patterns. AccessHub demonstrates account forms and client-side checks, Orbit demonstrates task-board interactions, and BusGo demonstrates a seat-selection and fare-summary workflow. They should be treated as prototypes until required back-end, security, persistence, and external integrations are implemented and tested.